ScoutOS Software Plan

Project Goal

ScoutOS is a minimalist motorcycle dashboard companion for a 2023 Indian Scout Bobber. The goal is to create a small circular display that shows only the information needed while riding: speed, turn-by-turn navigation, distance to next maneuver, ETA, and eventually optional motorcycle telemetry.

The first version should avoid replacing the stock gauge or reading motorcycle data. Instead, it should work as a Bluetooth display powered by a phone companion app.

Core Philosophy

ScoutOS should not be a tiny phone screen.

It should be a purpose-built riding interface:

- Large speed display
- One navigation instruction at a time
- Minimal visual clutter
- Fast boot
- Automatic reconnect
- Glanceable at speed
- Usable with gloves
- No dependency on the motorcycle ECU for V1

V1 Architecture

```
iPhone App
  |
  | Bluetooth Low Energy
  v
ESP32-S3 Round Display
  |
  v
LVGL Motorcycle UI
```

The phone handles complex tasks: GPS, routing, ETA, music metadata, internet access, settings, and updates.

The ESP32 handles simple tasks: receive state over BLE and render the UI.

Mobile App

Recommended stack:

- Flutter for app UI
- Native iOS bridge if needed for navigation SDK access
- Mapbox Navigation SDK for turn-by-turn routing
- Bluetooth Low Energy for device communication

Mapbox is the strongest feasibility choice because its iOS Navigation SDK supports custom navigation experiences and exposes turn-by-turn navigation logic; Google Maps and Waze do not cleanly expose live maneuver data to third-party hardware. Mapbox's Navigation SDK has "Active Guidance" trips for turn-by-turn sessions and "Free Drive" trips without a destination, so pricing/usage should be checked before committing long-term.

Firmware

Recommended stack:

- ESP-IDF
- LVGL for UI rendering
- BLE GATT server/client communication
- Optional Wi-Fi for OTA updates later

ESP32-S3 is a good target because it supports BLE 5, Wi-Fi, RGB/parallel LCD interfaces, and is commonly used for LVGL-style embedded display projects.

BLE Data Model

The phone should send a single versioned dashboard state object to the display.

Example:

```
{
  "version": 1,
  "timestamp": 1782942340,
  "ride": {
    "speedMph": 47,
    "headingDeg": 82
  },
  "navigation": {
    "active": true,
    "maneuver": "right",
    "distanceMeters": 640,
    "instruction": "Turn right",
    "streetName": "Main St",
```

```
    "etaMinutes": 14
  },
  "device": {
    "phoneBattery": 82,
    "gpsSignal": "good"
  }
}
```

For V1, JSON is fine. Later, this can be replaced with a compact binary protocol if needed.

Display Screens

Primary Ride Screen

Default screen while moving:

```
     47

     ↱

  0.4 mi

 Main St
```

Priority order:

1. Speed
2. Next maneuver arrow
3. Distance to maneuver
4. Street name
5. Connection or warning indicators

No Route Screen

```
     47

  No Route

 Connected
```

Waiting Screen

```
  ScoutOS
```

```
Waiting for
  phone...
```

Error Screen

```
Connection
   Lost

Reconnecting
```

Navigation Source Strategy

Preferred V1

Use Mapbox Navigation SDK.

Why:

- Provides navigation logic inside your app
- Supports custom UI
- Gives access to route/maneuver state
- Avoids screen scraping
- Avoids dependence on Beeline or another closed hardware ecosystem

Avoid for V1

Do not attempt to mirror Google Maps, Apple Maps, or Waze. They are not designed to expose live navigation instructions to custom motorcycle hardware.

Possible Future Alternative

Use an open routing stack:

- OpenStreetMap data
- Valhalla, GraphHopper, or OSRM routing
- Custom routing backend or offline routing

This would reduce vendor lock-in but is more work than Mapbox for V1.

MVP Requirements

The MVP is successful when:

- ESP32 display boots reliably
- Phone connects automatically over BLE

- App sends live GPS speed
- App sends next navigation instruction
- Display updates smoothly
- UI is readable while walking/driving test routes
- Reconnection works after power loss
- Display handles “no route,” “lost phone,” and “route ended” states

Non-Goals for V1

Do not include these yet:

- Motorcycle CAN bus integration
- Fuel level
- RPM
- Warning lights
- Touch-heavy UI
- Music controls
- Weather
- Full map display
- Replacing the stock gauge

These can come later after the core navigation display works.

Future V2 Features

Possible additions:

- Music metadata
- Incoming call indicator
- Weather/temp
- Favorite destinations
- Offline route caching
- OTA firmware updates
- Custom ride modes
- Trip stats
- Scout telemetry adapter
- Fuel/RPM/neutral/high beam/turn signal indicators

Long-Term Architecture

The display should be motorcycle-agnostic.

Internally, it should only care about a normalized dashboard state:

```
type DashboardState = {
  ride: RideState;
```

```
  navigation: NavigationState;
  media?: MediaState;
  vehicle?: VehicleState;
};
```

Later, motorcycle-specific adapters can provide vehicle data:

```
Phone Navigation Provider
        +
Scout Vehicle Adapter
        |
        v
Normalized Dashboard State
        |
        v
ESP32 Display
```

This keeps the UI reusable even if the bike-specific telemetry changes.

First Development Milestone

Build a desk prototype:

1. Flash ESP32-S3 display board
2. Render static LVGL screen
3. Add fake speed/navigation data
4. Add BLE connection
5. Build simple phone app that sends mock dashboard state
6. Replace mock data with phone GPS speed
7. Add Mapbox route state
8. Test in car before motorcycle use

Guiding Principle

The display should never try to be a phone.

It should be a beautiful, reliable, glanceable motorcycle instrument that shows only what matters right now.